Specificații detaliate, planul de proiect, prezența unui proiect în git

Funcționalități (minime)

- Aplicația are două feluri de utilizatori, studenți și profesori
- Un profesor are o serie de sesiuni de înscriere, care nu se pot suprapune
- În cadrul unei sesiuni de înscriere, un student poate face o cerere preliminară unui profesor
- Profesorul poate aproba cererea unui student în limita unui număr prestabilit
- Profesorul poate de asemenea respinge cererea cu o justificare
- Un student poate face adresa cereri mai multor profesori, dar dacă a fost aprobat de unul dintre ei, nu mai poate fi aprobat de un al doilea
- După aprobarea cererii preliminare, studentul poate încărca un fișier (cererea semnată)
- Ca răspuns la cerere, profesorul poate încărca un fișier sau respinge cererea, caz în care studentul va trebui să încarce o nouă cerere

Funcționalități adaugate

- Abilitatea de a filtra cererile (de către Profesor) sau profesorii (de către Student) după status (Approved, Pending) sau după nume.
- UI compatibil cu Mobile/Tablet
- Notificari in aplicatie
- Afisare procent locuri ocupate cu o bara de progres la profesori

Technologii folosite

Frontend: React, Vite, Javascript

Backend: Node.js, Sequelize, Postgres, Express

User			
Attribute	**Type**	**Constraints**	**Key Logic**
id	UUID	**PK**	Foundation for authentication.
email	String	**UNIQUE**, NOT NULL	Used for login.
password	String	NOT NULL	Password hash (cu bcrypt).
name	String	NOT NULL	Users full name.
role	Enum	NOT NULL ('Student' / 'Profesor')	Defines user permissions
Professor			
Attribute	**Type**	**Constrângeri & Relații**	**Logica Cheie**
professorId	UUID	**FK** User(id)	Extends the Professor role

max_students	Int	NOT NULL, Default: 0	The maximum number of students allowed for coordination.
current_students_count	Int	NOT NULL, Default: 0	Counter for occupied slots (managed by the Backend).
Student			
userId	String	**FK** User(id)	Extends the Student role
has_coordinator	Boolean	NOT NULL, Default: FALSE	TRUE if an approval request has been granted (enforces the Uniqueness Rule).
Session			
id	UUID	**PK**	Unique identifier for the enrollment session.
professorId	String	**FK** Professor(userId)	The professor owning the session.
data_ora_start	DateTime	NOT NULL	Start date and time of the session.

data_ora_sfarsit	DateTime	NOT NULL	End date and time of the session.
este_activa	Boolean	NOT NULL, Default: TRUE	Current status.
			Non-overlapping constraint must be checked in the backend logic (Express.js).
Dissertation Request			
id	UUID	**PK**	Unique identifier for the request.
studentId	String	**FK** Student(userId)	The student who submitted the request.
professorId	String	**FK** Professor(userId)	The professor requested.
sessionId	String	**FK** către Session(id)	The session the request was made during.
status	String	NOT NULL ('Pending',	Current state of the request.

		'Approved', 'Rejected')	
rejection_justification	String	NULL	Professor's explanation if the request is rejected.
url_student_file	String	NULL	Path/URL to the signed student document (File Upload).
url_professor_response	String	NULL	Path/URL to the professor's response file (File Upload).
			This table manages the **Single Approval Rule** and the **Document Flow**.